

NATIONAL ECONOMICS UNIVERSITY
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE & ARTIFICIAL INTELLIGENCE (FDA)

OFFICIAL PROGRESS EXAMINATION

Academic Year: 2026–2027

Semester: Fall 2026 Course Code:

DSAI1003

FUNDAMENTAL PROGRAMMING CONCEPTS IN PYTHON

PROGRESS EXAM: FUNCTIONS, MODULAR DESIGN & VARIABLE SCOPE

Paper Code: _____ | Duration: 60 Minutes (Excluding paper distribution)

Candidate Full Name:

Student ID (MSSV): ...

Class / Cohort:

Exam Room / Seat No.:

Section	A	B	C	D	Total (100)	Examiner Signature
Max Score	20	25	25	30	100	
Score						

Instructions: Closed-book examination. All electronic devices and internet access are prohibited. Answer all questions directly in the designated spaces. In programming tasks, apply modular programming principles, function definitions (def), parameter passing, return statements, and proper variable scope. Do **not** use global variables or third-party libraries. Organize scripts cleanly with helper functions and a designated main() driver. Do not detach pages.

MULTIPLE-CHOICE ANSWER SHEET (Write A, B, C, or D in the corresponding box):

Question	1	2	3	4	5	6	7	8	9	10
Answer										

Section A: Functions, Modular Design & Scope Concepts

(20 points)

Answer all 10 questions. Each correct answer carries 2 points. Suggested time: 12 minutes.

- Q1.** What is the central engineering principle behind treating a function as a “black box”?
- The caller must inspect every internal line of code before making a function call.
 - The caller relies only on the function’s interface (inputs, outputs, specification) without needing internal details.
 - A black-box function cannot accept parameters or return computed values to the caller.
 - It guarantees that the function executes with zero memory consumption.
- Q2.** What is the precise distinction between a **parameter** and an **argument** in Python?
- A parameter is the passed value; an argument is the variable declared in the function header.
 - Parameters and arguments are identical synonyms with no syntactic or conceptual distinction.
 - A parameter is the variable in the function header; an argument is the concrete value supplied in the call.
 - Arguments can only be string literals; parameters can only be numeric types.
- Q3.** What is the fundamental operational difference between `return value` and `print(value)`?
- `print()` passes data back to the calling expression; `return` only displays characters on screen.
 - `return` sends computed data back to the caller for further use; `print()` displays text and returns `None`.
 - `print()` halts the function immediately; `return` allows subsequent lines to execute.
 - Both statements are functionally identical and interchangeable in arithmetic expressions.
- Q4.** In Python, if a function finishes executing its body without a `return` statement, what does it return?
- The integer `0`.
 - An empty string `""`.
 - The special singleton value `None` of type `NoneType`.
 - A fatal compile-time `MissingReturnError` is raised.
- Q5.** Given `def modify(x): x = x + 10`, if a caller runs `a = 5` followed by `modify(a)`, what is `a`?
- 15, because integer arguments are passed by reference and modified in place.
 - 5, because parameter `x` is a local variable; reassigning `x` does not modify caller variable `a`.
 - 10, because the parameter replaces the caller’s variable in memory.
 - An `UnboundLocalError` occurs because `a` was not declared as global.

- Q6.** Which statement correctly defines the **lifetime** and **scope** of a local variable inside a function?
- It exists for the entire program duration and can be accessed from any function.
 - It is created when the function executes and destroyed upon exit; it cannot be referenced outside.
 - It retains its value across separate function calls without using the `global` keyword.
 - It is saved directly to disk and reloaded whenever any module is imported.
- Q7.** What happens when a local variable inside a function shares the same name as an existing global variable?
- Python raises a `NameCollisionError` when the script starts.
 - The global variable overrides the local variable, preventing the local variable from being assigned.
 - The local variable **shadows** (hides) the global variable within the function's local scope.
 - Python automatically merges both variables into a two-element tuple.
- Q8.** In the engineering process of **stepwise refinement** (top-down design), how should a complex problem be approached?
- Write all calculations sequentially inside one continuous global script without functions.
 - Decompose the complex problem into smaller sub-tasks, each implemented as an independent helper function.
 - Always implement recursion first before considering iterative or procedural decomposition.
 - Combine user inputs, validations, mathematical formulas, and screen formatting into a single function.
- Q9.** In software development, what is a function **stub**?
- A permanent syntax error that halts Python compilation.
 - A temporary placeholder implementation with a valid signature that returns a default value for incremental testing.
 - A built-in Python keyword used to terminate infinite loops.
 - A compiled binary extension written in C.
- Q10.** Why is organizing program logic within a `def main():` function considered a best practice in Python?
- It accelerates Python bytecode execution by 100 times.
 - It is mandatory because Python cannot run any statements outside a function named `main`.
 - It prevents accidental global variables, provides a clear program entry point, and promotes modular testing.
 - It automatically converts all input variables into floating-point numbers.

Section B: Output Tracing, Scope Diagnosis & Error Correction

(25 points)

Answer all 5 questions. Each question carries 5 points. Suggested time: 15 minutes.

Q11. Exact Output Tracing with Parameter Passing & Return Tuples (5 points)

Predict the **exact screen output**, including punctuation, spaces, and line breaks:

```
def compute_discount(subtotal, rate=0.10):
    discount = subtotal * rate
    return discount

def finalize_bill(amount, is_member):
    tax = amount * 0.08
    rate = 0.15 if is_member else 0.05
    disc = compute_discount(amount, rate)
    net_total = amount - disc + tax
    return disc, net_total

total = 100.0
discount, final_due = finalize_bill(total, True)
print(f"Subtotal: ${total:.2f} | Discount: ${discount:.2f}")
print(f"Final Due: ${final_due:.2f}")
print(f"Saved: {discount >= 15.0 and total == 100.0}")
```

Output Box for Q11

Q12. Evaluation of Function Calls and Return Values (5 points)

Write the exact evaluated value (or boolean result True / False) for each expression:

- def notify(msg): print(msg)
Result of notify("Done") is None: _____
- def cap(v, m=50): return m if v > m else v * 2
Result of cap(20) + cap(60): _____
- def valid(c): return len(c) == 6 and c[:2].isalpha() and c[2:].isdigit()
Result of valid("NE2026"): _____
- def step(x): x = x + 5; return x * 2 with a = 4; b = step(a)
Value of a + b: _____
- Evaluated result of max(round(3.456, 1), min(4.2, 3.8)): _____

Q13. Diagnose and Correct Common Function Defects (5 points)

For each code scenario, state the underlying defect and provide the corrected Python statement.

Code / Situation	Problem / Logic Flaw	Correction / Valid Code
def calc_area(w, h): print(w * h) ans = calc_area(5, 4) + 10		
def reset_counter(c): c = 0 count = 15 reset_counter(count)		
def prepare_total(): subtotal = 100.0 prepare_total() print(subtotal)		

Q14. Variable Scope, Name Shadowing & State Isolation (5 points)

Predict the exact screen output of the following script:

```
rate = 0.05

def apply_surcharge(base):
    rate = 0.12
    surcharge = base * rate
    return surcharge

def process_order(base):
    fee = apply_surcharge(base)
    total = base + fee + (base * rate)
    return total

amt = 200.0
due = process_order(amt)
print(f"Global Rate: {rate * 100:.0f}%")
print(f"Payable Due: ${due:.2f}")
print(f"Base Unchanged: {amt == 200.0}")
```

Output Box for Q14

Q15. Function Specification, Docstrings & Stubs (5 points)

- Write a standard Python **docstring** for a function calculate_fuel(distance, consumption_rate) explaining parameters, pre-conditions, and return value:

- Write a syntactically valid **function stub** for verify_token(token_str) that accepts a string and returns a safe default boolean value False:

Section C: Program Design & Modular Decomposition

(25 points)

Answer all tasks. Suggested time: 15 minutes.

Problem Specification: Logistics Fleet Trip Reimbursement Engine

A logistics dispatch platform calculates driver trip compensation based on travel distance d (km), cargo weight w (kg), driving duration t (hours), shift timing (`is_night`), and diesel fuel price per liter p :

- **Input Validation:** Distance must satisfy $d > 0$; cargo weight must satisfy $0 < w \leq 5000.0$; driving time must satisfy $t > 0$. Otherwise, reject as invalid.
- **Fuel Cost Computation:** Base fuel consumption is 10.0 L/100km. If cargo weight exceeds 2000.0 kg, heavy-load friction increases consumption by 15% (i.e. 11.5 L/100km). Fuel cost is: $\text{liters} \times p$, where $\text{liters} = (d \times \text{consumption_rate}) / 100.0$.
- **Driver Pay Calculation:** Driver receives a base wage of \$20.00/hour plus a mileage allowance of \$0.30/km. If `is_night` is True, the hourly rate receives a 25% premium (\$25.00/hour).
- **Total Reimbursement:** Sum of fuel cost and driver pay. If total trip distance exceeds 400.0 km, add a flat \$50.00 long-haul bonus.

Task 1: Identify Functions, Parameters & Return Types (6 points) Identify three independent helper functions suitable for this modular decomposition:

- **Function 1:** `validate_trip(d, w, t)` → Return Type: _____ | Purpose:
- **Function 2:** `compute_fuel_cost(d, w, p)` → Return Type: _____ | Purpose:
- **Function 3:** `compute_driver_pay(t, d, is_night)` → Return Type: _____ | Purpose:

Task 2: Write the Structured Modular Call Hierarchy in Ordered Steps (7 points)

Formulate sequential pseudo-logic showing how `compute_total_reimbursement` coordinates validation, invokes helper functions, applies conditional bonuses, and returns the net total.

Program-Logic Work Area

Task 3: Hand Trace / Step-by-Step Test Calculation (6 points)

For $d = 300.0$ km, $w = 2500.0$ kg, $t = 5.0$ hours, `is_night = True`, and fuel price $p = \$1.50/\text{L}$, calculate the fuel cost, driver pay, and final total reimbursement.

Calculation Box for Task 3

Task 4: Explain Two Software Engineering Design Decisions (6 points)

- Why must functions return calculated values rather than directly calling `print()` inside helper functions?
- Why is passing data via explicit parameters and return values superior to relying on shared global variables?

Section D: Applied Programming in Python

(30 points)

Write complete, syntactically correct Python code. Design well-structured modular functions with explicit parameters and return values. Do **not** use global variables or third-party libraries. Organize code with clean helper functions and a `main()` entry point. Suggested time: 18 minutes.

Problem 1: Modular Retail Inventory & Tiered Volume Billing System (15 points)

Write a modular Python program that computes customer order charges with volume discounting, weight-based shipping, and sales tax.

Requirements:

1. Implement `get_discount_rate(quantity)`: Returns a float discount rate based on units purchased: 0% if $\text{quantity} < 10$; 5% if $10 \leq \text{quantity} < 50$; 10% if $50 \leq \text{quantity} < 100$; 15% if $\text{quantity} \geq 100$.
2. Implement `compute_shipping(weight, is_express)`: Base fee is \$5.00 for $\text{weight} \leq 5.0$ kg. For weight exceeding 5.0 kg, add \$1.50 per excess kg. If `is_express` is True, add a flat \$12.00 express fee.
3. Implement `compute_order_total(unit_price, quantity, weight, is_express, tax_rate)`:
 - Gross subtotal = $\text{unit_price} \times \text{quantity}$. Discount amount = $\text{gross} \times \text{get_discount_rate}(\text{quantity})$.
 - Net merchandise = $\text{gross} - \text{discount}$.
 - Shipping fee: calculated via `compute_shipping(weight, is_express)`. **Free shipping rule:** if net merchandise $\geq \$200.00$ and `is_express` is False, shipping fee is waived (\$0.00).
 - Tax = $\text{net merchandise} \times \text{tax_rate}$. Total payable = $\text{net merchandise} + \text{shipping} + \text{tax}$.
 - Return a tuple: (`gross`, `discount`, `net_merchandise`, `shipping`, `tax`, `total_payable`).
4. Implement `main()`: Prompt user for inputs, validate positive price/quantity/weight, call `compute_order_total` with 8% tax rate (`0.08`), and display a formatted invoice.

Sample formatted output for \$25.00 unit price, 12 units, 8.0 kg, standard delivery (not express):

```
Gross Subtotal: $300.00
Volume Discount (5%): $15.00
Net Merchandise: $285.00
Shipping Fee (Waived): $0.00
Sales Tax (8%): $22.80
Total Payable: $307.80
```

Python Solution for Problem 1

Problem 2: Student Academic Standing & Merit Scholarship Evaluator (15 points)

Write a modular Python program to evaluate student academic performance and financial scholarship grants.

Requirements:

1. Implement `validate_student_record(student_id, gpa, credits)`: Returns True if `student_id` has length 8, starts with "NEU", ends with 5 digits (`student_id[3:].isdigit()`), $0.0 \leq \text{gpa} \leq 4.0$, and `credits` ≥ 0 ; otherwise False.
2. Implement `determine_standing(gpa, credits)`: Returns a standing string: "Dean's List" if `gpa` ≥ 3.60 and `credits` ≥ 15 ; "Good Standing" if `gpa` ≥ 2.50 ; "Academic Warning" if $2.00 \leq \text{gpa} < 2.50$; or "Academic Probation" if `gpa` < 2.00 .
3. Implement `calculate_scholarship(gpa, credits, is_financial_aid)`:
 - Merit tier: \$1200.00 if `gpa` ≥ 3.80 ; \$800.00 if $3.50 \leq \text{gpa} < 3.80$; \$400.00 if $3.20 \leq \text{gpa} < 3.50$; else \$0.00.
 - Need supplement: If `is_financial_aid` is True and `gpa` ≥ 2.50 , add \$350.00.
 - Full-time course grant: If `credits` ≥ 15 and initial scholarship $> \$0.00$, add \$150.00.
 - Return the total calculated scholarship amount as a float.
4. Implement `main()`: Prompt user for student details, validate via `validate_student_record`, and display a clean academic report card enclosed with border lines ("`=`" * 45).

Sample output for NEU54321, GPA 3.85, 16 credits, financial aid "yes":

```
=====
ACADEMIC STANDING & SCHOLARSHIP REPORT
-----
Student ID: NEU54321 | GPA: 3.85 | Credits: 16
Academic Standing: Dean's List
Merit & Need Scholarship: $1700.00
=====
```

Python Solution for Problem 2

— END OF EXAM PAPER —